

Evaluating a Deterministic Validator Plus Single-Iteration Repair Pipeline for LLM-Generated Network Configurations

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We preregistered and executed a paired confirmatory experiment (N=300 paired intent→configuration tasks; pre-registration id cand-2026-001-A-prereg-v1, pre-registration SHA-256 archived) to evaluate whether a minimal guarded pipeline—single-shot LLM generation (declared model: gpt-5-mini) followed by a frozen deterministic configuration validator and at most one automated repair iteration—reduces first-pass misconfiguration relative to plain single-shot generation. Execution completed 600 episodes and used 301 model calls. Observed outcomes: baseline = 299/300 valid (99.667%); guarded = 300/300 valid (100.0%); paired counts n11=299, n10=1, n01=0, n00=0. Exact McNemar two-sided $p = 1.00$; absolute paired difference = 0.0033333; paired bootstrap 95% CI = [0.00, 0.01] (10,000 resamples, seed 1729). The single permitted automated repair corrected the sole invalid baseline output (repair success 1/1; Clopper–Pearson 95% CI = [0.025, 1.0]). The preregistered planning assumption used for power calculations (planned baseline pass rate = 0.40) was contradicted by the observed near-ceiling baseline, removing statistical headroom for the preregistered $\geq 50\%$ relative reduction target. We report preregistered design choices, execution accounting, confirmatory statistics, a post-hoc inferential sensitivity bound tied to the observed contingency counts, and artifact-grounded recommendations for future NetOps confirmatory evaluations. All run artifacts and analysis outputs are archived in the execution manifest referenced in the Disclosure Statement.

I. INTRODUCTION

Translating operator intent to device configuration is a core NetOps task. Large language models (LLMs) have been proposed as intent→configuration translators that can accelerate operations, but probabilistic generation risks misordering, omission, and unsafe commands. Recent literature advocates hybrid patterns that pair probabilistic generators with deterministic validators/simulators and bounded repair loops as operational floor-safety nets. However, rigorous preregistered evaluations that quantify how tightly bounded guarded pipelines change first-pass misconfiguration rates under controlled sampling semantics are limited.

We implement and preregister a narrowly scoped paired experiment to evaluate the operational effect of a minimal guarded pipeline composed of: (1) a single shared LLM candidate per task (single-shot generation), (2) a frozen deterministic validator performing canonicalization and rule/dependency checks, and (3) at most one automated repair attempt when the validator reports violations. The core research question is: Can this minimal guarded pipeline reduce first-pass misconfiguration rate relative to plain single-shot generation under a

controlled paired design? Our contributions are (i) a preregistered paired experiment with deterministic seed generation and archived per-task artifacts, (ii) confirmatory statistics derived from those artifacts, (iii) a compact post-hoc sensitivity quantification tied to the observed contingency counts, and (iv) artifact-grounded recommendations for future NetOps confirmatory evaluations.

II. RELATED WORK

Recent work on LLMs for NetOps and intent-driven configuration has produced numerous system demonstrations and surveys documenting both promise and concrete failure modes. Empirical demonstrations of intent→configuration translation report that LLMs can synthesize plausible device configuration templates in lab and testbed settings, yet they frequently surface operationally relevant errors such as command misordering, omission of required safety steps, and overconfident assertions that lack mechanical verification (example system evaluation and discussion: Nguyen et al. 2024 [10.1002/nem.2313]). Complementary survey and review articles consolidate these observations and emphasize the need for grounded verification and governance when applying LLMs to critical network tasks [NIST guidance and surveys, 10.6028/nist.ai.100-2e2023].

Parallel technical threads motivate guarded architectures. Work in adjacent domains (clinical QA, code repair, and general RAG/verification research) shows measurable reductions in factual errors when probabilistic generators are paired with retrieval, deterministic checks, or repair loops; these findings have been ported by several NetOps proposals arguing for generate→verify→repair pipelines rather than blind single-shot deployment [e.g., comparative/procedural studies and framework discussions in the literature pool, 10.36227/techrxiv.176784505.56675782/v1; 10.2139/ssrn.6608518]. Architectural proposals for intent-aware automated remediation and multi-agent NetOps orchestration similarly argue that deterministic validators and bounded repair/backout policies are central to safe agentic NetOps workflows (see recent agentic NetOps proposals and intent-translation demonstrations, e.g., 10.36227/techrxiv.177004277.71795055/v1).

Deterministic network verification provides concrete algorithmic foundations for validators used in guarded pipelines. Prior work on specification-based and header-space analyses demonstrates tractable static checks for reachability, ac-

cess control, and update-safety properties without requiring live device execution; such algorithms can be integrated to canonicalize generated artifacts into normalized ASTs and to flag violations that trigger repair logic [Beckett et al., 2017 foundation for incremental/specification checks, 10.1145/3098822.3098834]. These deterministic checks differ fundamentally from empirical or emulation tests: they provide provable, rule-based pass/fail decisions for a defined specification, but they do not by themselves measure runtime control-plane or packet-level effects unless combined with higher-fidelity simulation or emulation.

Limitations in the prior literature motivate the present work. Many demonstrations and proposals are system-level or exploratory rather than preregistered confirmatory experiments; reported improvements from RAG, verifier-assisted generation, or agentic architectures often lack paired within-task statistical controls that isolate the incremental effect of a validator+repair floor on the same generated candidate. Moreover, published comparisons rarely quantify how sampling semantics (shared candidate versus independent draws) and repair budgets affect the number of informative discordant pairs available to paired binary tests. Surveys and architectural reviews therefore call for reproducible evaluation harnesses that combine deterministic validators with controlled sampling semantics and explicit inferential plans [10.6028/nist.ai.100-2e2023].

Our study addresses a tightly scoped portion of this gap by executing a preregistered paired experiment that (a) fixes `shared_initial_candidate` semantics to measure remediation capacity on a single generated plan, (b) uses a frozen deterministic validator to define a binary success criterion tied to intent constraints, and (c) limits repair budget to a single automated iteration. This design directly tests the operational claim that a minimal validator+repair floor reduces first-pass misconfiguration, and it surfaces the inferential consequences of high baseline pass rates and shared-candidate pairing for confirmatory McNemar-style testing.

III. METHODOLOGY

Preregistration and confirmatory hypotheses: The experiment was preregistered under id `cand-2026-001-A-prereg-v1` before any final outcomes were observed. Two confirmatory hypotheses were registered: H1 — the guarded pipeline reduces first-pass misconfiguration rate by $\geq 50\%$ (relative) versus single-shot generation; H2 — one or fewer automated repair iterations recover $\geq 75\%$ of initially invalid configurations. The preregistration explicitly documented the numeric planning assumption used for power calculations: planned baseline pass rate = 0.40. This planning assumption informed the targeted sample size and the preregistered effect magnitude; because it is central to the inferential design we report it here in the Methods for transparent interpretation.

Design and sampling semantics: We used a paired within-task design with a fixed deterministic seed list of 300 tasks produced by a frozen task generator. Each seed produced an intent and an associated canonical reference

configuration; seeds were included deterministically unless a preregistered parser exclusion rule applied (such replacements are recorded in the archived manifest). For each seed the same single LLM candidate was used in both arms (`shared_initial_candidate` semantics): the generator produced one configuration (shared candidate), which served as the baseline response and was the starting point for the guarded arm’s validation and possible repair. Planned episodes = 600 (two episodes per seed), `initial_generation_calls_per_task` = 1, `maximum_repair_calls_per_task` = 1. The shared-candidate pairing was preregistered to isolate the incremental effect of deterministic validation and bounded repair on a single proposed plan; as noted below, this choice increases within-pair correlation and reduces the number of statistically informative discordant pairs available to paired tests compared with an independent-draws design.

Model, resource constraints, and provider-call policy: The declared generation model in the execution contract was `gpt-5-mini`. The execution contract limited total model calls to a planned maximum of 600 (one initial generation per seed across 300 seeds plus up to one repair per seed). Provider call failures are handled per the preregistered retry policy (single retry allowed for provider/infrastructure failures); the primary missingness treatment drops pairs only if both calls in a pair failed (`'complete_pair_primary'`). These operational constraints are part of the preregistration because they affect the realized information and the primary analysis set.

Validator canonicalization, rules, and scoring semantics: The frozen deterministic validator implemented canonicalization to a normalized configuration AST and applied a suite of static checks derived from explicit workflow and dependency policies encoded in the task payloads. The scoring rule used across the experiment was full intent-constraint validation: a generated configuration is scored as success only when (a) the validator’s `normalized_configuration` satisfies the task’s `required_sequence` and `expected_final_state` constraints, and (b) all preregistered workflow policies (e.g., `'must_be_down_for_fields'`, `'final_group_constraints'`, and `'minimum_active_in_groups'`) are satisfied. The validator’s trace records `parsed_command_count`, `required_command_count`, `observed_touched_field_count`, `violation_codes`, and a boolean valid flag; these elements were the operational primitives used to trigger repair and to compute per-task scores. Canonicalization enables deterministic AST equality and normalized-string comparisons used in contamination detection and exact matching checks.

Repair adapter behavior and steering: The preregistered automated repair adapter is invoked only when the validator reports violations on the shared candidate. The repair adapter receives the shared candidate plus structured violation codes and is restricted to a single automated repair attempt per episode. Its objective is minimal, targeted editing of the shared candidate sufficient to remove the reported violations while preserving as much of the original plan as permitted by task constraints. Repair prompts and repair responses were

archived per episode; the repair outcome was revalidated by the same deterministic validator to produce the final guarded decision. Because the repair budget was intentionally bounded to one call, repair-effectiveness estimands are conditional on the initially invalid subset and must be interpreted with that budget constraint in mind.

Contamination checks, exclusions, and replacement rules: The preregistration included deterministic contamination detection based on exact normalized-AST equality and deterministic near-duplicate lexical thresholds; any exact duplicates between task targets and transformation/unit-test artifacts would trigger deterministic exclusion and deterministic replacement from a preregistered reserve set. The primary analysis uses the decontaminated paired set produced by these rules; sensitivity analyses are specified to report results on the full set when flags arise. All exclusions, replacements, and contamination flags are recorded in the archived manifest per the preregistered plan.

Primary estimand and statistical analysis plan: The primary estimand was the `paired_success_rate_difference_guarded_minus_baseline` (absolute difference in per-task success probability under guarded vs baseline). The preregistered primary test was the exact McNemar two-sided test at $\alpha=0.05$ on paired binary outcomes; paired bootstrap percentile 95% CIs for the absolute paired difference were preregistered (10,000 resamples, explicit seed). Secondary analyses included exact binomial CIs for repair success conditional on initial failure, stratified paired checks by task difficulty and constrained vendor templates, and sensitivity treatments for failed provider calls (treat as failures or exclude per the preregistered missingness plan). The preregistration contained a power/precision plan that translated the planned baseline pass rate = 0.40 and a target relative reduction ($\geq 50\%$ relative reduction in failure rate) into a required N under conservative discordant assumptions; that planning arithmetic is reported in the preregistration record and is discussed in the manuscript’s interpretation because the observed baseline diverged from that assumption.

Sampling-semantics tradeoffs and inferential consequences: We preregistered the `shared_initial_candidate` semantics to align the experiment with operational workflows that commonly evaluate and remediate a single proposed plan. This design reduces cross-arm generation variance (the same generated plan is evaluated and potentially repaired) but increases within-pair correlation: paired discordance counts are limited to cases where the validator+repair pipeline changes the acceptability of that single candidate. Consequently, the number of informative discordant pairs available to McNemar inference is typically smaller than under independent-draws designs and must be accounted for when translating planned effect sizes into target sample sizes. The preregistration explicitly encoded this tradeoff in its power calculations; we reiterate it here to make the methodological rationale and its inferential implications explicit in the Methods rather than only in the Discussion.

Reproducibility and archival primitives: The experiment preregistered and archived all deterministically generated seeds, the task generator configuration, the scoring rules, validator trace formats, repair prompt templates, the analysis code, and the execution manifest. Key reproducibility primitives include the normalized AST canonicalization used by the validator (the basis for deterministic equality checks), the pair sampling semantics (shared candidate per pair), the maximum repair budget (one per episode), the missingness/retry policy, and the primary exact McNemar test with bootstrap CI parameters (10,000 resamples, explicit seed). These artifacts are enumerated in the execution manifest and analysis bundle referenced in the Disclosure Statement and constitute the authoritative sources for the numeric counts and inferential inputs reported in this paper.

IV. RESULTS

Execution accounting (artifact-derived): Planned episodes = 600; completed episodes = 600; complete pairs analyzed = 300. Model calls used = 301 (300 shared initial generations + 1 repair invocation). No provider or infrastructure failures occurred under the preregistered retry policy; no deterministic exclusions for contamination were required under the preregistered checks. Per-task normalized responses, validator decisions, and repair traces are archived and are the direct source of the numerical counts below.

Primary outcomes (archived counts): `Baseline_success_count` = 299/300 (99.667%); `Guarded_success_count` = 300/300 (100.0%). Paired contingency (guarded vs baseline) derived from archived analysis artifacts: $n_{11} = 299$ (both success), $n_{10} = 1$ (guarded success, baseline fail), $n_{01} = 0$ (guarded fail, baseline success), $n_{00} = 0$ (both fail). These counts are reproduced in the archived paired contingency artifact used for confirmatory inference.

Inferential results (preregistered): Exact McNemar two-sided test on the observed contingency gives $p = 1.00$. The observed absolute paired difference (guarded - baseline success rate) = $1/300 \approx 0.003333$. The paired bootstrap percentile 95% CI for the absolute paired difference = $[0.00, 0.01]$ (10,000 resamples, bootstrap seed = 1729). Repair outcomes (H2): the single allowed automated repair corrected the sole invalid baseline output (repair success 1/1); Clopper-Pearson 95% CI for 1/1 is $[0.025, 1.0]$, indicating wide uncertainty given the single observed repair.

Post-hoc sensitivity bound (artifact-grounded): The total number of discordant pairs observed is $n_{10} + n_{01} = 1$. Under the paired sampling semantics used, this discordant total imposes a hard limit on the resolvable absolute paired difference equal to $1/300 \approx 0.00333$. Therefore (a) the maximal absolute improvement consistent with the observed discordant count is $1/300$; and (b) the preregistered target corresponding to a $\geq 50\%$ relative reduction from a planned baseline of 0.40 ($\approx$ absolute +0.30) would have required on the order of 90 net discordant pairs favoring guarded outputs. This artifact-grounded bound explains why the preregistered large relative effect was

not detectable: the empirical baseline prevalence (299/300) left essentially no statistical headroom for the planned effect size. The archived contingency artifact and bootstrap results support these numerical statements.

Representative diagnostic artifacts: For each episode the archived validator trace contains `normalized_configuration`, `parsed_command_count`, `required_command_count`, `violation_codes`, and `repair_applied` flags. The single baseline failure was traceable in those artifacts and was corrected by invoking the preregistered repair adapter on the shared candidate; the post-repair canonicalized configuration satisfied the full intent constraints according to the deterministic validator. These representative artifacts are available in the archived execution bundle described in the Disclosure Statement.

V. DISCUSSION

The guarded pipeline demonstrably remediated the single invalid baseline output observed in this run: the deterministic validator flagged the violation and the single permitted automated repair produced a canonicalized configuration that the validator accepted. This outcome empirically supports the practical utility of a frozen validator plus a tightly bounded repair step as a floor-safety mechanism that can convert some validator-detected failures into valid first-pass configurations.

However, the run also illustrates an important inferential and design caveat: confirmatory power for paired tests depends critically on the number of informative discordant pairs. Our preregistered `shared_initial_candidate` pairing was chosen to model operational workflows that examine and remediate a single generated plan. That design reduces within-pair heterogeneity and isolates the effect of validation+repair on a given candidate, but it also reduces the expected discordant pair count relative to independent draws per arm. In our execution the baseline pass rate was 299/300, producing only one discordant pair and thereby imposing a strict numerical ceiling on resolvable absolute improvement ($1/300 \approx 0.00333$). Consequently, the preregistered $\geq 50\%$ relative-reduction target—formulated under a planning assumption `baseline_pass_rate = 0.40`—became unattainable given the observed prevalences. The archived preregistration explicitly records the `baseline_pass_rate = 0.40` planning assumption and the analysis artifacts show the realized counts; together they explain why the confirmatory hypothesis could not be supported despite the guarded pipeline correcting the single failure it encountered.

Operationally useful lessons flow directly from these artifact-grounded facts. First, when baseline error prevalence is uncertain, researchers should design task banks or sampling procedures to produce an expected number of initial failures (or discordant pairs) aligned with the planned detectable effect. For the preregistered aim in this study, the analysis artifacts indicate that on the order of 90 discordant pairs would have been needed to resolve an absolute change comparable with the preregistered target (≈ 0.30 absolute). In practice, investigators can (a) increase task difficulty, (b) include adversarial or stress-test augmentations, or (c) adopt stratified sampling by

difficulty to concentrate statistical information where it is most informative. These are operational design options rather than claims about guaranteed outcomes; their selection should be preregistered and justified against expected discordant counts.

Second, choice of pairing semantics must be matched to the operational claim. If the goal is to measure whether validator+repair improves the probability that an operator will accept a generated plan when the generator is sampled afresh per condition, independent per-arm draws increase discordance and statistical information for McNemar-style inference. If, instead, the workflow examines and remediates a single proposed plan (the `shared_initial_candidate` semantics used here), then evaluation budgets should anticipate reduced discordant counts and compensate via deliberately higher-difficulty tasks or larger N . The preregistration tradeoff—higher within-pair control versus greater discordant information—should be made explicit in power computations; our archived power plan assumed `baseline_pass_rate = 0.40` and thus did not account for the realized near-ceiling baseline.

Third, repair-effect estimands are inherently conditional on initial failures and require sufficient counts of those failures to estimate repair effectiveness precisely. In this run the observed repair success was 1/1, which is artifact-grounded but statistically imprecise (Clopper–Pearson 95% CI = [0.025, 1.0]). When repair effectiveness is a primary operational claim, studies should be powered to observe an adequate number of initial failures so that conditional repair success has narrow confidence intervals; appropriate sample sizing depends on the target precision and should be preregistered. The deterministic artifacts archived here (per-task validator traces and repair prompts) exemplify the level of per-case traceability required to support conditional estimands and subsequent diagnostics.

Finally, construct and external validity remain constrained in this study. Our validator performed deterministic static canonicalization and rule checks rather than full control-plane or packet-level emulation; therefore, runtime semantics and emergent control-plane interactions were not exercised. The task bank is synthetic and vendor-template coverage was constrained, and only one declared model (gpt-5-mini) and one validator implementation were used. These limitations narrow direct production generalization but do not diminish the methodological point: artifact-grounded preregistration combined with deterministic validator traces yields reproducible counts that clarify what was actually tested and which inferential questions remain open.

In sum, the empirical artifact shows that small guarded pipelines can correct individual validator-detected errors, but proving large relative reductions in misconfiguration rate under paired designs requires aligning task difficulty, pairing semantics, and sample size with the planned estimand. We therefore recommend that future confirmatory NetOps evaluations (a) preregister pairing semantics and the expected discordant-pair count implied by the power calculation, (b) calibrate task difficulty or use adversarial augmentation to achieve a target initial-failure prevalence when necessary, and (c) archive per-task validator and repair traces to support transparent

conditional estimands and post-hoc diagnostics. The archived execution and analysis artifacts accompanying this study provide a concrete example of how those recommendations can be operationalized.

VI. LIMITATIONS

- Synthetic task bank and constrained vendor-template scope limit external validity to broad production environments.
- Single LLM (gpt-5-mini) and a single validator implementation restrict generality across model families and validator designs.
- The preregistered power plan assumed a baseline pass rate = 0.40; the observed baseline (299/300) contradicts that assumption and removed headroom to detect the preregistered $\geq 50\%$ relative reduction.
- Sampling semantics (shared_initial_candidate) reused the same initial candidate across paired arms, increasing within-pair correlation and reducing discordant pairs available to McNemar inference.
- Validation used deterministic configuration/constraint checks rather than full dynamic emulation of runtime semantic effects; actual runtime consequences of configurations were not executed and remain unmeasured.
- H2 repair-success evidence is based on a single initially invalid case (1/1 $\rightarrow$ 100%), yielding a wide Clopper-Pearson 95% CI = [0.025, 1.0]; larger counts of initial failures are required to estimate repair effectiveness precisely.

VII. CONCLUSION

We executed a preregistered paired experiment (N=300 pairs) comparing single-shot LLM generation (gpt-5-mini) to a guarded pipeline consisting of a frozen deterministic validator and a single automated repair iteration. Baseline single-shot generation yielded 299/300 valid outputs and the guarded pipeline yielded 300/300 valid outputs. The single automated repair corrected the lone invalid baseline output, but the observed near-ceiling baseline contradicted the preregistered planning assumption (baseline pass rate = 0.40) and removed statistical headroom to detect the preregistered $\geq 50\%$ relative reduction. Future confirmatory NetOps evaluations should (a) calibrate task difficulty to produce sufficient initial failures or target discordant counts, (b) preregister pairing semantics and repair budgets explicitly, and (c) include emulation to measure runtime semantic effects when operational deployment claims are intended. All raw outputs, validator traces, repair traces, the execution manifest, and analysis artifacts are archived and enumerated in the Disclosure Statement to permit reproduction of the reported counts and intervals.

DISCLOSURE STATEMENT

This study was executed autonomously after the autonomy lock under the archived master prompt (provenance/master_prompt.txt; SHA-256: 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba).

Human role: the Research Director selected the topic family (Generative AI and LLMs for NetOps) and approved the master prompt prior to the autonomy lock; no manual human editing of manuscript technical content, figures, tables, or references occurred after the lock. Preregistration: cand-2026-001-A-prereg-v1 (the preregistration record was created before observing final outcomes; preregistration SHA-256 is recorded in the execution manifest). Declared model and tooling: gpt-5-mini (declared_model_version), adapter family hosted_netops_gvr_v1, frozen deterministic validator/repair adapter deterministic_netops_validator_v5. Execution accounting (archived): planned episodes = 600, completed episodes = 600, complete pairs = 300, model_calls_used = 301 (300 shared initial generation calls + 1 repair invocation), repair-stage invocations = 1. The execution manifest and analysis artifact bundle enumerate all raw model responses, per-task validator traces (normalized configurations, parsed_command_count, required_command_count, violation_codes, repair_applied flags), repair prompts/responses, execution logs, and analysis artifacts. The archived execution manifest and analysis bundle are the authoritative source for the numeric counts reported in this manuscript. The autonomous pipeline performed literature search, preregistration, execution, analysis, AI-peer-review style checks, and manuscript drafting autonomously after the autonomy lock in accordance with the CNSM Agentic AI Researcher track requirements. The preregistered planning assumption used in power calculations (planned baseline pass rate = 0.40) is explicitly reported here because it materially affects interpretation of the confirmatory result.

REFERENCES

- [1] Nguyen Tu, Sukhyun Nam, James Won-Ki Hong, "Intent-Based Network Configuration Using Large Language Models", 2024, 10.1002/nem.2313
- [2] Oghenekeno Hilkiyah Okula, "The Era of AI Agents for Network Engineering": Intent-Aware Auto Remediation for Network Disruption or Failures Using AI Agents, Large Language Models (LLM) and Model Context Protocol (MCP) Mechanisms", 2026, 10.36227/techrxiv.177004277.71795055/v1
- [3] Kunal Dhanda, "MedRAG: Retrieval-Augmented Generation for Medical QA-Comparing Base and RAG-Augmented LLM Performance on Evidence from Peer-Reviewed Clinical Research", 2026, 10.2139/ssrn.6608518
- [4] Goutam Adwant, Manjari Srivastav, "Evidence Coverage Evaluator: A Comprehensive Framework for Assessing Grounding Quality in Retrieval-Augmented Generation Systems", 2026, 10.36227/techrxiv.176784505.56675782/v1
- [5] Ryan Beckett, Aarti Gupta, Ratul Mahajan, David Walker, "A General Approach to Network Configuration Verification", 2017, 10.1145/3098822.3098834
- [6] Apostol Vassilev, Alina Oprea, Alie Jean Fordyce, Hyrum S. Anderson, "Adversarial machine learning :", 2024, 10.6028/nist.ai.100-2e2023